

Django with Abstract Base Classes & Composition

[Using DjangoGetting Started](#)

You have selected **0** posts.

[select all](#)

[cancel selecting](#)

6.6k views

7 likes

6 links



[4](#)



[3](#)



1 / 8

post by guzmanojero on Dec 3, 2022



[guzmanojero](#)

1

[Dec 2022](#)

I'm following this Python tutorial from ArjanCodes that talks about composition over inheritance and I wonder how to implement it in Django.



Composition Is Better Than Inheritance in Python

[Composition Is Better Than Inheritance in Python](#)

I would like to implement the example used (see code below) and I wonder how to implement it in Django. I'm thinking in applying the SOLID principles using interfaces.

The questions are related to Django Models.

1. How to use Python's abc module to implement interfaces.
2. How to use composition instead of inheritance.

1. How to use Python's abc module to implement interfaces.

I would like to design the model as an interface, having the implementation in the subclasses, like this:



```
class Contract(ABC):
    """Represents a contract and a payment process for a particular employeee."""

    @abstractmethod
    def get_payment(self) -> float:
```

```
"""Compute how much to pay an employee under this contract."""
```

```
@dataclass
class HourlyContract(Contract):
    """Contract type for an employee being paid on an hourly basis."""

    pay_rate: float
    hours_worked: int = 0
    employer_cost: float = 1000

    def get_payment(self) -> float:
        return self.pay_rate * self.hours_worked + self.employer_cost
```

I don't know if the abstract model class (`abstract = True`) can do this.
I've read this answers:

- [python 3.x - Inheriting from both ABC and django.db.models.Model raises metaclass exception - Stack Overflow](#)
- [python - django models - how can i create abstract methods - Stack Overflow](#)

They propose a solution:

```
import abc

from django.db import models

class AbstractModelMeta(abc.ABCMeta, type(models.Model)):
    pass

class AbstractModel(models.Model, metaclass=AbstractModelMeta):
    # You may have common fields here.

    class Meta:
        abstract = True

    @abc.abstractmethod
    def must_implement(self):
        pass

class MyModel(AbstractModel):
    code = models.CharField("code", max_length=10, unique=True)

    class Meta:
        app_label = 'my_app'
```

But I don't know if this is a good solution.

2. How to implement composition instead of using inheritance.

It seems that you can apply composition using relationships within your models (like OneToOne).

Is there a different way? Like using Dependency Injection in the traditional sense?

Like this:

```
@dataclass
class Employee:
    """Basic representation of an employee at the company."""
```

```

name: str
id: int
contract: Contract # ***Dependency Injection here***
commission: Optional[Commission] = None

def compute_pay(self) -> float:
    """Compute how much the employee should be paid."""
    payout = self.contract.get_payment()
    if self.commission is not None:
        payout += self.commission.get_payment()
    return payout

henry_contract = HourlyContract(pay_rate=50, hours_worked=100)
henry = Employee(name="Henry", id=12346, contract=henry_contract)

```

The full example code is here:

[with_composition.py](#)

[main](#)

```

"""
Very advanced Employee management system.
"""

from abc import ABC, abstractmethod
from dataclasses import dataclass
from typing import Optional

class Contract(ABC):
    """Represents a contract and a payment process for a particular employee."""

    @abstractmethod
    def get_payment(self) -> float:
        """Compute how much to pay an employee under this contract."""

@dataclass
class Commission(ABC):
    """Represents a commission payment process."""

```

This file has been truncated. [show original](#)

Sorry if this is too long.
Thanks.

- [How to use SOLID principles in Django](#)

6.6k views

7 likes

6 links



[4](#)



[3](#)



post by KenWhitesell on Dec 4, 2022

post by guzmanojero on Dec 4, 2022

post by KenWhitesell on Dec 4, 2022

post by guzmanojero on Dec 4, 2022

post by KenWhitesell on Dec 4, 2022

10 months later

post by MrCordeiro on Sep 29, 2023

post by KenWhitesell on Sep 30, 2023

Reply

Related topics

Topic list, column headers with buttons are sortable.

Topic	Replies	Views	Activity
Combining Django's django.db.models.Model + abc.abstractmethod	15	2.8k	Jun 2021
Getting Started			
How to use SOLID principles in Django	1	769	Jun 2024
Getting Started			
Abstract models as mixins?	2	122	May 2025
Using the ORM			
Opinions on a few choices I made with my app	10	1.4k	Jul 2021
Using Django			
Abstract base model for abstracting particular save behavior	11	3.7k	Aug 2021
Using Django			

Invalid date

Invalid date